

maceR Developer Guide

Architecture, Workflow, and Component Integration

Marco Ferrari

14 April, 2026

Contents

1	Introduction	4
1.1	Quick Facts	4
2	Part 1: Application Architecture	4
2.1	High-Level Overview	4
2.1.1	Core Design Pattern	4
2.2	Directory Structure	4
3	Part 2: State Management with R6 Classes	5
3.1	Overview	5
3.2	Key R6 Classes	5
3.2.1	1. App_settings (fct_R6_App_settings.R)	5
3.2.2	2. Raw_mouse_data (fct_R6_Raw_mouse_data.R)	5
3.2.3	3. Clean_mouse_data (fct_R6_Clean_mouse_data.R)	6
3.2.4	4. Custom_tables (fct_R6_Custom_tables.R)	6
3.2.5	5. Annotate (fct_R6_Annotate.R)	6
4	Part 3: Data Flow & Workflow	6
4.1	Complete User Journey	6
4.1.1	Step 1: Data Upload (mod_Input_DF)	6
4.1.2	Step 2: Data Structure Check (mod_data_structure)	7
4.1.3	Step 3: Subsetting (mod_your_data)	7
4.1.4	Step 4: Analysis (mod_analysis)	7
4.1.5	Step 5: Visualization (mod_plots)	7
4.1.6	Step 6: Report Generation (mod_report)	7

5	Part 4: Module Structure	8
5.1	What is a Module?	8
5.1.1	Module Naming Convention	8
5.2	Standard Module Template	8
5.3	Key Modules in maceR	9
6	Part 5: Adding New Functionality	9
6.1	Scenario 1: Add a New Plot Type	9
6.1.1	Step 1: Add Method to Annotate R6 Class	9
6.1.2	Step 2: Add Handler in Analysis Module	10
6.1.3	Step 3: Add UI Control	10
6.1.4	Step 4: Register in Output List	10
6.2	Scenario 2: Add a New Analysis Tab	10
6.2.1	Step 1: Create Module File	11
6.2.2	Step 2: Add to Main App	11
6.3	Scenario 3: Add Data Export Format	11
6.3.1	Step 1: Install Package	12
6.3.2	Step 2: Create Export Function	12
6.3.3	Step 3: Add Download Button	12
7	Part 6: Adding to Report Generation	12
7.1	Integrating New Content into Reports	12
7.1.1	Step 1: Collect Data in mod_report	13
7.1.2	Step 2: Update Report Template	13
8	Part 7: Common Patterns & Best Practices	13
8.1	Pattern 1: Reactive Data Flow	13
8.2	Pattern 2: Accessing R6 Objects from Modules	14
8.3	Pattern 3: Loading Dynamic Plots	14
8.4	Pattern 4: Error Handling in Reports	14
9	Part 8: Debugging & Development Tips	14
9.1	Setting Up Development Environment	14
9.2	Using browser() for Debugging	15
9.3	Inspecting R6 Objects	15
9.4	Common Issues	15

10 Part 9: Quick Reference	16
10.1 Creating a New Component Checklist	16
10.2 File Organization	16
11 Conclusion	16

1 Introduction

This guide is designed for developers new to the maceR codebase. It provides an overview of the application architecture, explains the core workflow, and demonstrates how to integrate new components and functionalities into the system.

1.1 Quick Facts

- **Framework:** Shiny (via Golem boilerplate)
 - **Architecture:** Modular with R6 state management
 - **Primary Use:** Interactive analysis of circadian locomotor activity data
 - **Language:** R
 - **Key Dependencies:** behavr, ggetho, zeitgebr, ggplot2, data.table, dplyr
-

2 Part 1: Application Architecture

2.1 High-Level Overview

The maceR application follows a **modular Shiny architecture** with centralized state management using **R6 classes**. Data flows through a series of transformations, from raw upload to visualization and reporting.

2.1.1 Core Design Pattern

```
User Input (UI)
↓
Module Server Logic (Reactivity)
↓
R6 State Objects (Persistent Storage)
↓
Analysis Functions (Computation)
↓
Visualization (Plots & Reports)
↓
Download / Export
```

2.2 Directory Structure

```
maceR/
  R/
    app_ui.R           # Main UI definition
    app_server.R       # Main server orchestration
    mod_*.R            # Feature modules (UI + Server)
    fct_*.R            # Utility & computation functions
    fct_R6_*.R         # R6 class definitions
    utils_*.R          # Helper functions
    run_app.R          # App startup
  inst/
```

app/www/	# Static assets (CSS, JS)
rmarkdown/	# Report templates
man/	# Documentation files
data-raw/	# Raw data for examples
DESCRIPTION	# Package metadata

3 Part 2: State Management with R6 Classes

3.1 Overview

R6 classes are the backbone of maceR's state management. They store application state in a persistent, accessible way across different modules and sessions.

3.2 Key R6 Classes

3.2.1 1. App_settings (fct_R6_App_settings.R)

Purpose: Central configuration and environment holder

Key Fields: - `dashboardHeader_appearance`: UI theme toggle - `dataList`, `metadata`: File upload paths - `listMice`: Lists of processed animals - `subsetting`: Filter parameters - `LDcondition`, `LDparams`: Light cycle parameters - `plotTab`: Visibility flags for tabs - `env1`, `env2`, `env3`, `env4`: Environment pointers (see below) - `env_msg`: Messages object environment

Key Methods: - `setData()`, `setMeta()`: Store uploaded files - `setListMice()`: Process mouse ID list - `setListMiceFiltered()`: Apply filtering - `setLD()`: Configure light cycle

Where It's Stored:

App_settings (created in `app_server.R`)

- `env1` → `myMice` environment (`Raw_mouse_data` objects)
- `env2` → `myCleanMice` + `Custom_tables` environments
- `env3` → `Custom_tables` environment
- `env4` → `Annotate` environment (plots stored here)

3.2.2 2. Raw_mouse_data (fct_R6_Raw_mouse_data.R)

Purpose: Store raw data from uploaded files

Key Fields: - `id`: Animal identifier - `metadata`: Raw metadata row - `timepoint`: Time vector - `countsMinute`: Activity counts

Key Methods: - `add()`: Load and parse data files

Storage: Created in `preload_data()` (`fct_data_loader.R`), stored in `env$env1$myMice`

3.2.3 3. Clean_mouse_data (fct_R6_Clean_mouse_data.R)

Purpose: Cleaned, transformed data ready for analysis

Key Fields: - Individual animal data with converted time formats - `sex`, `genotype`, `cabinet`: Metadata fields - `lightOn`: Light-on hour - `length`: Number of datapoints

Key Methods: - `compile()`: Transform raw data - `addData()`: Populate fields

Storage: Created in `load_data()`, stored in `env$env2$myCleanMice`

3.2.4 4. Custom_tables (fct_R6_Custom_tables.R)

Purpose: Aggregated data suitable for visualization

Key Fields: - `locomotor_act`: Lists of activity data at different aggregations - `daily_act`: Daily activity summaries - `periodograms`: Circadian period data - `average_day`: Average daily patterns

Key Methods: - `compile()`: Create tables from clean data - `locomotor_subset()`, `daily_subset()`: Filter functions

Storage: Created in `load_data()`, stored in `env$env3$Custom_tables`

3.2.5 5. Annotate (fct_R6_Annotate.R)

Purpose: Store generated plots and metadata

Key Fields: - `Actograms`: Single-plotted actogram list - `DPActograms`: Double-plotted actogram list - `DAct_plots`: Daily activity plots - `period_plots`: Periodogram plots - `period_plots_box`: Period distribution plots - `avg_day_plots`: Average day plots - `Report`: Generated report data (NEW) - `metaTable`: Animal metadata - `actTable`: Currently displayed activity table

Key Methods: - `showMeta()`: Display metadata - `showData()`: Display animal data - `plot_actogram()`: Generate actogram - `plot_double_actogram()`: Generate double-plotted actogram - (and many others for different plot types)

Storage: Created in `load_data()`, stored in `env$env4$Annotate`

4 Part 3: Data Flow & Workflow

4.1 Complete User Journey

4.1.1 Step 1: Data Upload (mod_Input_DF)

```
User uploads CSV files + metadata
↓
mod_Input_DF_server() validates and stores paths
↓
App_settings$setData() & setMeta()
↓
preload_data() creates Raw_mouse_data objects
↓
Raw data stored in env1
```

4.1.2 Step 2: Data Structure Check (mod_data_structure)

User reviews data structure
↓
mod_data_structure_server() calls load_data()
↓
load_data() creates:
- myCleanMice list (env2)
- Custom_tables (env3)
- Annotate object (env4)

4.1.3 Step 3: Subsetting (mod_your_data)

User selects animals/filters
↓
mod_your_data_server() captures selections
↓
App_settings\$setListMiceFiltered()
↓
Filtered list stored in App_settings\$subsetting

4.1.4 Step 4: Analysis (mod_analysis)

User selects plot types & parameters
↓
mod_analysis_server() triggers analyses
↓
Calls Annotate\$plot_*() methods
↓
Plots stored in Annotate\$ fields
↓
Observes trigger plot loading

4.1.5 Step 5: Visualization (mod_plots)

load_plots() called with plot list
↓
Creates UI boxes dynamically
↓
Renders plots from Annotate
↓
User can download individual plots

4.1.6 Step 6: Report Generation (mod_report)

User selects report options
↓
mod_report_server() collects all plots
↓
Stores in Annotate\$Report
↓

User clicks download
↓
rmarkdown::render() with parameters
↓
Pandoc generates HTML or PDF
↓
Browser downloads file

5 Part 4: Module Structure

5.1 What is a Module?

A Shiny module is a reusable piece of UI and server logic. maceR uses modules extensively to keep code organized and maintainable.

5.1.1 Module Naming Convention

- **UI Function:** `mod_<name>_ui(id)`
- **Server Function:** `mod_<name>_server(id, ...)`
- **File Location:** `R/mod_<name>.R`

5.2 Standard Module Template

```
# UI Function
mod_example_ui <- function(id) {
  ns <- NS(id)
  tagList(
    shinydashboardPlus::box(
      title = "Example Module",
      width = 12,
      # UI elements here
      actionButton(ns("action"), "Do Something")
    )
  )
}

# Server Function
mod_example_server <- function(id, env) {
  moduleServer(id, function(input, output, session) {
    ns <- session$ns

    # Reactive values
    example_data <- reactiveVal(NULL)

    # Observers & event handlers
    observeEvent(input$action, {
      # Logic here
    })
  })
}
```



```

    example_data(new_data)
  })

  # Return reactive outputs that other modules can use
  return(reactive({
    list(data = example_data(), success = TRUE)
  }))
})
}

```

5.3 Key Modules in maceR

Module	Purpose	Key Return
mod_landing_page	Welcome screen	None
mod_input_DF	File upload	Trigger signal
mod_data_structure	Data validation	Trigger for next step
mod_your_data	Data inspection	None
mod_analysis	Plot parameter selection	Plot triggers
mod_plots	Plot display	None
mod_report	Report generation	Download handler

6 Part 5: Adding New Functionality

6.1 Scenario 1: Add a New Plot Type

Goal: Create a new analysis plot (e.g., “Average hourly activity”)

6.1.1 Step 1: Add Method to Annotate R6 Class

File: R/fct_R6_Annotate.R

```

Annotate <- R6::R6Class("Annotate",
  list(
    # ... existing fields ...
    avg_hourly_plots = list(AvgHour1 = list()),

    # New method
    plot_average_hourly = function(env, type) {
      data <- env$env3$Custom_tables$locomotor_act[[1]]
      # calculation logic
      plot <- ggplot2::ggplot(data) +
        # plot specification
        ggplot2::ggtitle("Average Hourly Activity")

      self$avg_hourly_plots$AvgHour1[[1]] <- plot
    }
  )

```

```
)  
)
```

6.1.2 Step 2: Add Handler in Analysis Module

File: R/mod_analysis.R

```
observeEvent(input$avg_hourly_checkbox, {  
  if (input$avg_hourly_checkbox) {  
    Annotate <- App_settings$env4$Annotate  
    Annotate$plot_average_hourly(App_settings$env3, type = "total")  
    # Trigger plot loading  
    load_plots(env = App_settings,  
              plot_list = "avg_hourly",  
              session = session)  
  }  
})
```

6.1.3 Step 3: Add UI Control

File: R/mod_analysis.R (UI function)

```
checkboxInput(  
  ns("avg_hourly_checkbox"),  
  "Average Hourly Activity",  
  value = FALSE  
)
```

6.1.4 Step 4: Register in Output List

File: R/fct_R6_Annotate.R

Add to output_list_acto tibble:

```
# ... in output_list_acto data frame  
"hourlyAvg" # handler  
"AvgHour1" # destination  
"Annotate$avg_hourly_plots$AvgHour1[[1]]" # location  
"Average Hourly Activity" # title  
# ... and other metadata columns
```

6.2 Scenario 2: Add a New Analysis Tab

Goal: Create a new analysis workflow (e.g., “Statistical Comparison”)

6.2.1 Step 1: Create Module File

File: R/mod_statistics.R

```
mod_statistics_ui <- function(id) {  
  ns <- NS(id)  
  tagList(  
    shinydashboardPlus::box(  
      title = "Statistical Comparison",  
      width = 12,  
      selectInput(ns("stat_type"), "Test:",  
                  choices = c("t-test", "ANOVA")),  
      actionButton(ns("run_test"), "Run Analysis")  
    )  
  )  
}  
  
mod_statistics_server <- function(id, env) {  
  moduleServer(id, function(input, output, session) {  
    observeEvent(input$run_test, {  
      data <- env$env3$Custom_tables$locomotor_act[[1]]  
      # Perform statistical analysis  
      results <- your_analysis_function(data, input$stat_type)  
      # Store or display results  
    })  
  })  
}
```

6.2.2 Step 2: Add to Main App

File: R/app_ui.R

```
# In the shinydashboardPlus::dashboardBody  
menuItem("Statistics", tabName = "Statistics", icon = icon("chart-bar"))
```

File: R/app_server.R

```
# Create the tab when analysis is complete  
observeEvent(analysis_out$plotTab(), {  
  output$Statistics <- render_tabItem_ui(tabname = "Statistics",  
                                          text = "Statistics",  
                                          icon = "chart-bar")  
})  
  
# Call module server  
mod_statistics_server("stats_ui_1", App_settings)
```

6.3 Scenario 3: Add Data Export Format

Goal: Allow exporting data as Excel files

6.3.1 Step 1: Install Package

```
# In DESCRIPTION, add to Imports:
openxlsx
```

6.3.2 Step 2: Create Export Function

File: R/utils_converters.R

```
export_to_excel <- function(data_list, filename) {
  wb <- openxlsx::createWorkbook()

  for (sheet_name in names(data_list)) {
    openxlsx::addWorksheet(wb, sheetName = sheet_name)
    openxlsx::writeData(wb, sheet_name, data_list[[sheet_name]])
  }

  openxlsx::saveWorkbook(wb, filename)
}
```

6.3.3 Step 3: Add Download Button

File: R/mod_your_data.R (or wherever appropriate)

```
output$download_excel <- downloadHandler(
  filename = function() {
    paste0("maceR_data_", format(Sys.Date(), "%Y%m%d"), ".xlsx")
  },
  content = function(file) {
    data_list <- list(
      metadata = App_settings$env4$Annotate$metaTable,
      activity = App_settings$env4$Annotate$actTable
    )
    export_to_excel(data_list, file)
  }
)
```

7 Part 6: Adding to Report Generation

7.1 Integrating New Content into Reports

The report system is parameterized through the `Annotate$Report` field.

7.1.1 Step 1: Collect Data in mod_report

File: R/mod_report.R

```
# In the generate_report observer
plots_data <- list(
  # ... existing plots ...
  new_analysis = env$env4$Annotate$new_analysis_plots,
  statistics_results = env$env4$Annotate$stat_results # NEW
)
```

7.1.2 Step 2: Update Report Template

File: inst/rmarkdown/templates/maceR_report/skeleton/skeleton.Rmd

```
# New Analysis Section

\\`\\`{r new-analysis}
if (!is.null(params$plots_data$new_analysis)) {
  for (i in seq_along(params$plots_data$new_analysis)) {
    if (!is.null(params$plots_data$new_analysis[[i]])) {
      print(params$plots_data$new_analysis[[i]])
    }
  }
}
\\`\\`
```

8 Part 7: Common Patterns & Best Practices

8.1 Pattern 1: Reactive Data Flow

```
# Good: Chain reactives for clarity
user_selection <- reactive({ input$selected_ids })
filtered_data <- reactive({
  Custom_tables$filter_by_ids(user_selection())
})
results <- reactive({
  compute_analysis(filtered_data())
})

# Less clear: All logic in one reactive
results <- reactive({
  # multiple steps mixed together
})
```

8.2 Pattern 2: Accessing R6 Objects from Modules

```
# Recommended: Pass entire environment
mod_analysis_server <- function(id, env) {
  moduleServer(id, function(input, output, session) {
    # Access via env$env4$Annotate
  })
}

# Alternative: Pass App_settings directly
mod_analysis_server <- function(id, App_settings) {
  moduleServer(id, function(input, output, session) {
    # Access via App_settings$env4$Annotate
  })
}
```

8.3 Pattern 3: Loading Dynamic Plots

```
# Use the load_plots() utility function
load_plots(env = App_settings,
           plot_list = c("plot_type1", "plot_type2"),
           session = session)

# This creates UI boxes and renders plots dynamically
```

8.4 Pattern 4: Error Handling in Reports

```
# Template code
\\`\\`{r section, error=TRUE}
if (!is.null(params$plots_data$analysis)) {
  tryCatch({
    print(params$plots_data$analysis[[1]])
  }, error = function(e) {
    cat("Error:", conditionMessage(e), "\n")
  })
} else {
  cat("No analysis available")
}
\\`\\`
```

9 Part 8: Debugging & Development Tips

9.1 Setting Up Development Environment

```

# Load all code and dependencies
devtools::load_all()

# Run tests
devtools::test()

# Check package integrity
devtools::check()

# Run the app
maceR::run_app()

```

9.2 Using browser() for Debugging

```

# In any server function or reactive
observeEvent(input$action, {
  browser() # Execution pauses here
  # Inspect App_settings, env$env4$Annotate, etc.
})

```

9.3 Inspecting R6 Objects

```

# View structure
str(App_settings$env4$Annotate)

# Check specific field
App_settings$env4$Annotate$Actograms

# Verify method
head(App_settings$env4$Annotate$metaTable)

```

9.4 Common Issues

Issue	Solution
“Plot not displaying”	Check <code>load_plots()</code> call and verify plot in Annotate
“NULL reference error”	Ensure prerequisite steps completed (data loaded, analysis run)
“Module not rendering”	Check module call in <code>app_server.R</code> with correct ID
“Report won’t download”	Verify params declared in template YAML
“File upload fails”	Check file format matches expected CSV structure

10 Part 9: Quick Reference

10.1 Creating a New Component Checklist

- ☐ Define R6 class or add method to existing class
- ☐ Add UI controls in module
- ☐ Add server logic to handle UI events
- ☐ Update `App_settings` or state object if needed
- ☐ Register in main app (`app_ui.R`, `app_server.R`)
- ☐ Test with `devtools::load_all()` and `run_app()`
- ☐ Add documentation/comments
- ☐ Include in report if applicable

10.2 File Organization

Data handling: `R/fct_R6_*.R` and `R/fct_data_loader.R` **UI components:** `R/mod_*.R` (UI section)
Business logic: `R/mod_*.R` (server section) **Utilities:** `R/utils_*.R` and `R/fct_*.R` **App entry point:** `R/app_ui.R`, `R/app_server.R`, `R/run_app.R` **Reports:** `inst/rmarkdown/templates/`

11 Conclusion

The maceR application provides a solid foundation for circadian activity analysis with a clear architecture that supports easy extension. Key takeaways:

1. **R6 objects** handle all persistent state
2. **Modules** keep UI and logic organized
3. **Data flows** through predictable stages (`raw` → `clean` → `tables` → `plots`)
4. **Reports** automatically include generated visualizations
5. **New features** can be added by extending R6 classes and creating new modules

For questions about specific implementations, refer to the inline code comments in each file and the main server/UI logic in `app_server.R` and `app_ui.R`.

Document Version: 1.0

Last Updated: 14 April, 2026

Target Audience: Developers new to maceR codebase